FlightSim + HHMARL_3D_NEW

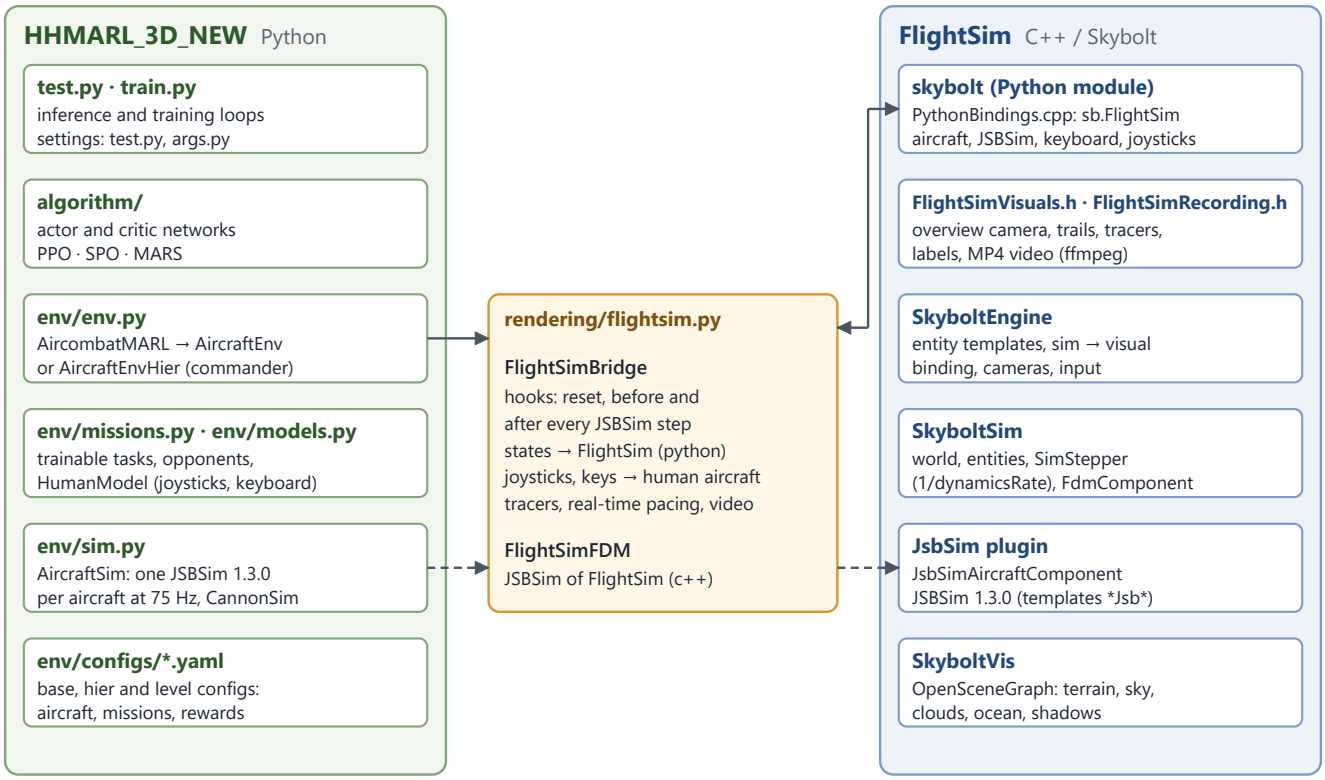
Components, architecture and parameters

Status 27 October 2026 · FlightSim Python API version 8 · JSBSim 1.3.0 at 75 Hz

1 Overview

- **HHMARL_3D_NEW**: multi-agent air combat environment in Python on JSBSim, with its own on-policy MARL training (MAPPO, MASPO, MARS). Training always runs here, without FlightSim.
- **FlightSim**: a modified Skybolt (C++, OpenSceneGraph) with the Python module `skybolt`. It is used for inference only: realistic rendering, video recording, optionally the C++ JSBSim, and a human pilot with joysticks or the keyboard.
- The repositories are separate clones side by side (`Projects/FlightSim`, `Projects/HHMARL_3D_NEW`). HHMARL imports FlightSim only when it is used.
- The HHMARL env always computes observations, rewards and terminations. FlightSim displays the aircraft, reads the joysticks and the keyboard or simulates the flight dynamics.

2 Architecture



Solid arrows: always used with FlightSim. Dashed: only with `JSBSIM_STATES = "c++"`, where AircraftSim calls the JSBSim of FlightSim.

One env step with FlightSim

1. The policies act. Each mission turns its action into JSBSim flight controls (aileron, elevator, rudder, throttle) and fires the cannon.
2. `sim_steps` (10) JSBSim steps of 1/`sim_freq` s (1/75 s) follow. Before each step, FlightSimBridge hides aircraft that were shot down or crashed, writes the controls of the human (joysticks or keyboard) and, in `c++` mode, advances the JSBSim of FlightSim (`step(1)`). After each step, it copies the states of the Python JSBSim into FlightSim (python mode, `step(1)`), shows tracers while a cannon fires and renders in real time (`speed`). While recording, every frame of the video is rendered, even if the simulation then runs slower than real time.
3. The env computes observations, rewards and terminations, as without FlightSim.

Commander (`hier`): one env step assigns a low-level policy (cannon, track or escape) to each aircraft and runs it for `hl_sim_steps` low-level steps. With FlightSim, all aircraft advance in lockstep (all set their controls, then all step together).

JSBSim modes

JSBSIM_STATES	Flight dynamics	FlightSim templates
`"python"` (default)	jsbsim Python package of HHMARL; FlightSim only displays the states	`SepecatJaguarBlue`, `SepecatJaguarRed`
`"c++"`	JSBSim plugin of FlightSim; `FlightSimFDM` replaces `jsbsim.FGFDMExec` in AircraftSim	`SepecatJaguarJsbBlue`, `SepecatJaguarJsbRed` (JSBSim model `f16` only)

Both modes use JSBSim 1.3.0 at 75 Hz and give identical trajectories; `Tools/Compare_JSBSim_Python_Cpp.py` checks this.

3 FlightSim

3.1 Components

Component	Where	Role
skybolt Python module	`src/Skybolt/SkyboltPythonBindings/PythonBindings.cpp`	`sb.FlightSim`: window, aircraft, JSBSim access, cameras, keyboard, labels
Joysticks	`src/Skybolt/SkyboltPythonBindings/FlightSimJoysticks.h`	joysticks, throttles and pedals of the manual aircraft (`Tools/joysticks.json`), read with SDL 2.32.10
Visuals	`src/Skybolt/SkyboltPythonBindings/FlightSimVisuals.h`	overview camera (F3), trails, cannon tracers
Recording	`src/Skybolt/SkyboltPythonBindings/FlightSimRecording.h`	MP4 video of the window view (key R), encoded by ffmpeg in a background thread
SkyboltSim	`src/Skybolt/SkyboltSim`	world, entities and components; `SimStepper` (fixed steps of 1/dynamicsRate); `FdmComponent` (flight dynamics interface)
JSBSim plugin	`src/Skybolt/SkyboltEnginePlugins/JsbSim`	JSBSim 1.3.0 for templates with a `jsbSimAircraft` component
SkyboltEngine	`src/Skybolt/SkyboltEngine`	engine root, entity templates, sim-to-visual binding, camera controllers, keyboard and mouse
SkyboltVis	`src/Skybolt/SkyboltVis`	OpenSceneGraph rendering: planet, terrain, sky, clouds, ocean, shadows
Assets	`Assets/`	templates (`Movers/Entities`), models, F-16 of the manual aircraft (`Movers/JSBSimManual`), shaders (`Core`), planet and map tiles (`Globe`, `mapbox_*_ch`)
Dependencies	`conanfile.py`, `Conan/Recipes`	Conan 2 packages, incl. the JSBSim 1.3.0 recipe and SDL 2.32.10 (joysticks)
Path setup	`flightsim_env.py`	`import flightsim_env` before `import skybolt`: finds the build, sets library and asset paths and `FLIGHTSIM_FFMPEG`, reads `Tools/mapbox_token.txt`, sets `FLIGHTSIM_JOYSTICKS`, selects the dedicated GPU (Windows: graphics preference of python.exe; WSL2, Linux: environment variables of Mesa and NVIDIA)
Examples	`Test_FlightSim_Windows.py` `Test_FlightSim_Linux.py`	manual flight (joysticks or keyboard) of a JSBSim aircraft next to an aircraft moved from Python

3.2 Python API: sb.FlightSim

Constructor argument	Default	Meaning
`x, y, width, height`	`0, 0, 1080, 720`	window position and size in pixels
`manual`	`False`	the joysticks (3.3) or the keyboard fly the aircraft of `set_manual_aircraft(id)`
`multiwindow`	`False`	second window with a free camera
`dynamicsRate`	`75.0`	simulation steps per second
`headless`	`False`	no window, e.g. for comparisons (not with manual or multiwindow)

Area	Calls and properties
Aircraft	`add_aircraft(id, template, position, rpy, velocityNed)` · `set_aircraft_state(id, position, rpy, velocityNed=None)` · `remove_aircraft(id)` · `set_aircraft_visible(id, visible)` · `set_dynamics_enabled(id, enabled)` · `get_aircraft_ids()` · `get_aircraft(id)`

Area	Calls and properties
Units	`position`: `sb.LatLonAlt(lat, lon, alt)`, geodetic radians, m above sea level · `rpy`: `sb.Vector3` roll, pitch, yaw in radians (north-east-down) · `velocityNed`: north, east, down in m/s
JSBSim	`get_property(id, name)` · `get_properties(id, names)` · `set_property(id, name, value)` · `query_property_catalog(id)` · `run_ic(id)` · `get_num_engines(id)` · `init_engine_running(id, engine)` · `get_steady_state(id)` · `set_external_control(id, enabled)`: flight controls only from the properties, not from the joysticks or the keyboard
Simulation	`step(substeps=1)` · `render()`: `False` once the window is closed or Esc is pressed · `frame(dt)`: steps for `dt` seconds and renders · `sim_time` · `dynamics_step_size`
Window	`vsync` (key V, on: render() waits for the refresh of the monitor) · `fullscreen` (key B; a change of the window size stops a recording)
Camera	`select_camera_view(view)` with `"Cockpit"`, `"Follow"`, `"Overview"` or `"Free"` · `set_camera_target(id)`
Labels, trails	`set_aircraft_label(id, text)` · `set_aircraft_color(id, (r, g, b))` (linear RGB) · `labels_visible` · `trails_visible` · `trail_length` (m, 30000) · `clear_trails()` · `set_aircraft_throttle(id, throttle)`: nozzles of the camera target, without JSBSim
Cannon	`fire_tracer(id, speed=1000.0, lifetime=1.0)`: dark tracer line · `clear_tracers()`
Recording	`start_recording(path=None)`: returns the video file · `stop_recording()` · `recording` · `recording_frame_due` · `recording_directory` (`"recordings"`) · `recording_fps` (30), see 3.6
Manual flight	`set_manual_aircraft(id)` · `get_manual_controls()`: aileron, elevator, rudder, throttle · `consume_fire_requests()`: presses of the fire button or Space since the last call · `fire_pressed`: held
Low level	`engineRoot` · `world` · `visRoot` · `window` · `camera` · `sb.captureScreenshot(app.visRoot)`

3.3 Keyboard, mouse and joysticks

Key	Action
F1 · F2 · F3 · F4	cockpit · follow (external) · overview of all aircraft · free camera
Tab	next aircraft as camera target
L · T · R	labels · trails · start/stop recording a video
V · B	vsync on/off (shown at the top of the window and in the help) · full screen on/off
H · Esc	help · exit
Mouse, wheel	rotate the camera; in the overview, rotate around the aircraft and zoom with the wheel
A/D · W/S	overview: rotate · zoom (unless they fly the manual aircraft); free camera: W/S move
Manual flight	without joysticks: Up/Down pitch down/up · Left/Right roll · A/D rudder · W/S throttle · Space cannon
Joysticks	the devices of `Tools/joysticks.json` fly the manual aircraft while they are connected, each control otherwise with its keys: stick (roll, pitch), throttle lever, rudder pedals, trigger (fires while held). Device names and axis and button numbers as shown by `Tools/joysticks.py`, optionally `invert` and `deadzone`; read when FlightSim starts, so changes need no rebuild

3.4 Setup, tools and settings

Item	Parameters and use
`setup_windows.bat`	full setup; `-BuildDir <folder>`, `-PythonExe <python.exe>`
`setup_linux.sh`	full setup; `--build-dir <folder>`, `--python <python3.11>`, `--jobs N`
`Tools/update_pythonbindings_windows.bat` `Tools/update_pythonbindings_linux.sh`	rebuild after C++ changes (Linux: `--jobs N`). Run the full setup instead after adding or removing source files or changing dependencies.
`Tools/Compare_JSBSim_Python_Cpp.py`	flies an F-16 in both JSBSims for 120 s at 75 Hz (`SECONDS`, `RATE`) and prints the differences
`Tools/Tile_Map_Downloader.py`	downloads (or unzips from `Assets`) the offline map tiles of Switzerland; `--zoom 15 16` for more detail (needs a Mapbox token); area: `CENTER_LATITUDE`, `CENTER_LONGITUDE` in the script
`Tools/mapbox_token.txt`	optional Mapbox access token (Git stores only the comments, filter set up by the setup). Without it, only the offline map tiles are shown; with it, the missing tiles are loaded from Mapbox
`Tools/joysticks.json`	device and axis or button of each flight control of the manual aircraft: roll, pitch, throttle, rudder, fire (3.3)

Item	Parameters and use
`Tools/joysticks.py`	shows the names and the axes, buttons and hats of the connected joysticks, throttles and pedals
`Tools/Scale_OSG_Files.py`	scales a model, e.g. by `8.0` for the enlarged *Big models; `--to-osgt` converts it to text
`Tools/build_instructions_pdf.py`	builds this `instructions.pdf` into the current folder (Windows fonts)
Packages of .venv	installed by the setup: `conan` 2.22.0, `jsbsim` 1.3.0, `requests`, `imageio-ffmpeg` 0.6.0 (ffmpeg of the videos), `reportlab` 5.1.0 (this PDF), `pygame-ce` 2.5.8 (`Tools/joysticks.py`)
Environment variables	`FLIGHTSIM_BUILD_DIR`: build folder (default `../FlightSimBuild`) · `FLIGHTSIM_DIR`: FlightSim folder for HHMARL (default `../FlightSim`) · `FLIGHTSIM_FFMPEG`: ffmpeg of the videos (default: the one of imageio-ffmpeg, otherwise ffmpeg on PATH) · `FLIGHTSIM_MAPBOX_TOKEN`: Mapbox token (default: the one of `Tools/mapbox_token.txt`) · `FLIGHTSIM_JOYSTICKS`: joystick mapping (default `Tools/joysticks.json`, empty: keyboard only)
`Settings.json`	Windows `%LOCALAPPDATA%\Skybolt`, Linux `~/Skybolt`: `display` (`multiSampleCount`, `vsync`, `texturePoolSizeMB`), `shadows`, `clouds`, `mouse.sensitivity`, `tileApiKeys` (keep empty, the Mapbox token is in `Tools/mapbox_token.txt`), `jsb.rootDir` (JSBSim aircraft data)
`PlanetEarth.json`	`Assets/Globe/Entities`: `maxLevel` of the map tiles; raise it after downloading more zoom levels; `fallback`: online Mapbox tiles, used with a token
Templates	`Assets/Movers/Entities`: `SepecatJaguarBlue/Red` (moved from Python), `SepecatJaguarJsbBlue/Red` (JSBSim `f16`), `SepecatJaguarJsbBlue/RedManual` (F-16 with corrected rudder for the manual aircraft, `aircraftDir`), `SepecatJaguarBlueBig/RedBig` (enlarged models)

3.5 Visual constants (C++)

Element	Values in `FlightSimVisuals.h` (rebuild after changing)
Overview camera	vertical field of view 0.6 rad, looks down at 12°, margin 1.15 around the aircraft, at least 60 m away, zoom factor up to e^3
Trails	a point every 25 m, about 3 px wide, opacity 0.9 fading quadratically along `trail_length`
Tracers	20 m dark line, about 4 px wide, starting 10 m ahead of the aircraft
Nozzles	discs at the nozzle exits of the camera target (`nozzles` of the entity template), linearly from black at idle to dark red (linear RGB 0.045, 0.0012, 0) at full throttle (`fcs/throttle-cmd-norm`)

3.6 Video recording

Aspect	Behavior
Start, stop	key R or `start_recording()` / `stop_recording()`; also stops when the window is closed or the `FlightSim` is deleted; a red dot in the top left corner shows the recording
Content	the view of the window (selected camera with labels, trails and cockpit HUD if shown), without the help message and the red dot; size of the window (rounded down to even pixels)
File	`<recording_directory>/flightsim_<date>_<time>.mp4`, or the path given to `start_recording(path)`; a relative folder is relative to the current directory
Frame rate	`recording_fps` (30) frames per second of simulation time, each from the simulation step closest to its time, so the video plays at simulation speed (at 75 Hz alternately 2 and 3 steps apart)
Skipped rendering	programs that skip frames to keep up with real time must still render when `recording_frame_due` is `True`, otherwise frames repeat (FlightSimBridge does)
Encoding	H.264 (libx264, CRF 18, yuv420p, BT.709) by the ffmpeg of `FLIGHTSIM_FFMPEG`; not with `headless`

4 HHMARL_3D_NEW

4.1 Components

Module	Role
`train.py`	training; `process_args` applies experiment-specific overrides
`test.py`	inference over many episodes (win and loss rates), with rendering
`args.py`	all arguments of training and inference (command-line flags)
`algorithm/`	learner (PPO, SPO, MARS), actor and critic networks, value normalization
`env/env.py`	`AircombatMARL` (interface of the learner) → `AircraftEnv` (low level) or `AircraftEnvHier` (commander)
`env/sim.py`	`AircraftSim` (one JSBSim per aircraft) and `CannonSim`
`env/missions.py`	trainable tasks: track, cannon, escape, hier
`env/models.py`	scripted and pre-trained opponents, `HumanModel` (joysticks or keyboard via FlightSim)
`env/rewards.py`, `env/terminations.py`	reward terms and episode ends
`env/configs/`	`base.yaml`, `hier.yaml`, `levels/<task>_L<n>.yaml` (deep-merged)
`env/base_actor/`, `env/catalog.py`	scripted base actor and JSBSim property catalog from LAG
`rendering/`	`render.py`, `scene.py`: PyGame renderer · `flightsim.py`: FlightSim bridge
`runs/`	checkpoints and TensorBoard logs: `runs/<task>_<model>_<run_name>`
`recordings/`	videos recorded with key R in the FlightSim window (ignored by git)

4.2 Inference settings (test.py)

Setting	Default	Meaning
`RENDER`	`True`	render while testing
`RENDERER`	`"flightsim"`	`"pygame"` (simple) or `"flightsim"`
`RESTORE_PATH`	`runs\cannon_L4_f16`	checkpoint file, or run folder (its latest checkpoint)
`TASK`, `LEVEL`	`"cannon"`, `4`	task and level of the policy; commander: `--hierarchy_level 2`
`INIT_STATES`	`None`	per aircraft, relative to the battle field center: `north`/`east` in m (or `lat`/`lon` in degrees), `alt` in m, `speed` in m/s, `heading`, `roll`, `pitch` in degrees; `None`: config
`JSBSIM_STATES`	`"python"`	`"python"` or `"c++"`, see section 2
`MANUAL`, `MANUAL_AGENT`	`False`, `"B2"`	a human flies `MANUAL_AGENT` with the joysticks or the keyboard: a copy of B1 if the config has no such aircraft (track, cannon, escape), otherwise it replaces the RL agent (hier: B3)
`LOCATION`	`(8.965318, 45.975305)`	battle field center (longitude, latitude), here Lugano
`run_inference(num_episodes)`	`500`	number of episodes

4.3 FlightSim bridge (env config "flightsim")

Options of `FlightSimBridge` in `rendering/flightsim.py`. `test.py` passes `jsbsim_states` and `manual_agent`; add further options to `kwargs["flightsim"]` there.

Option	Default	Meaning
`jsbsim_states`	`"python"`	`"python"` or `"c++"`
`manual_agent`	`None`	ID of the aircraft flown by the human, e.g. `"B2"`
`window`	`(10, 30, 1600, 900)`	x, y, width, height of the window
`speed`	`1.0`	simulated seconds per real second; `None`: as fast as possible
`camera_view`	`None`	`"Cockpit"`, `"Follow"`, `"Overview"` or `"Free"`; `None`: cockpit with a human, otherwise overview
`trail_length`	`30000.0`	length of the trails in m
`templates`	`None`	templates per team, e.g. `{"R": "SepecatJaguarRedBig"}`; with c++, they need a `jsbSimAircraft` component

Option	Default	Meaning
`tracer_interval`	`5`	JSBSim steps between tracers while a cannon fires
`headless`	`False`	no window (not with manual_agent)
`recording_dir`	`None`	folder of the videos of key R; `None`: `recordings/` of HHMARL_3D_NEW. While recording, every video frame is rendered (no frames are skipped), so a slow PC runs slower than real time
`TEAM_COLORS`	`B (0.05, 0.3, 1.0)` `R (1.0, 0.06, 0.04)`	module constant: colors of the labels and trails per team (linear RGB)
`FLIGHTSIM_API_VERSION`	`8`	module constant: minimum version of the FlightSim build (8: `set_aircraft_throttle` for the nozzles)

4.4 Environment (AircombatMARL(task, **kwargs))

Argument	Values	Meaning
`task`	`"track/L4"`, `"cannon/L1"`, `"escape/L2"`, `"hier"`	config: `base.yaml` + level file or `hier.yaml`
`hierarchy_level`	`1` or `2`	1: low-level policies, 2: commander
`training_level`	integer	stage of the random initial states (set by train.py and test.py)
`rendering`, `render_view`, `render_samples`	`False`, `0`, `4`	PyGame rendering, viewpoint, anti-aliasing samples
`flightsim`	`None`	options of 4.3; `None`: without FlightSim
`config_overrides`	`None`	deep-merged into the config, e.g. `{"battle_field_center": [lon, lat, 0.0]}`
`reset(seed, options)`	`{"init_states": ...}`	initial states as `INIT_STATES`

4.5 YAML configuration (env/configs)

Key	Default	Meaning
`sim_freq`	`75`	JSBSim steps per second
`sim_steps`	`10`	JSBSim steps per env step (0.133 s)
`hl_sim_steps`	`10` (hier)	low-level steps per commander step
`horizon`	`600`; hier `100`	env steps per episode
`map_size`	`50`; hier `80`	arena size in km; farther than map_size/2 from the center is a crash
`altitude_limit`, `altitude_limit_max`	`2500`, `10000`	allowed altitudes in m
`acceleration_limit_x/_y/_z`	`9.0`	overload limit in g
`rnd_init`	`true`	random initial heading, position, altitude and speed
`battle_field_center`	`[0.0, 0.0, 0.0]`	longitude, latitude, altitude (test.py: `LOCATION`)
`num_rl_agents`, `num_opp_agents`	hier `3`, `3`	team sizes of the commander config
`opp_hier_strategy`	`mixed`	commander opponents: `mixed` (cannon/track/escape 40/40/20 %), `aggressive` (70/20/10 %), `random`
`sp_policy_name`	L3 levels	policy of the self-play opponent, e.g. `track_L2_f16`
`aircraft.<ID>`	`B1`, `R2`, ...	first letter = team; `model` (`f16` or `A4`), `mission` (4.6), `num_missiles` (cannon shots, 10), `init_state` (`ic_long_gc_deg`, `ic_lat_geod_deg`, `ic_h_sl_ft`, `ic_psi_true_deg`, `ic_u_fps`), `unlimited_fuel` (`true`)
`<Reward>_scale`, `<Reward>_potential`	`1.0`, `false`	per reward term: `EventDrivenReward`, `ShootReward`, `ShootPenaltyReward`, `HeadingReward`, `DistanceReward`, `FleeReward`; also `ShootReward_orientation_kill`, `ShootPenaltyReward_posture_punish`, `HeadingReward_track`, `HeadingReward_opposite`

4.6 Missions and interfaces

Mission (`aircraft.<ID>.mission`)	Behavior
`task_track`, `task_cannon`, `task_escape`	trainable low-level skills: pursue, attack with the cannon, evade
`task_hier`	trainable commander: picks `cannon_L4`, `track_L4` or `escape_L2` per aircraft
`model_L1Maneuver`, `model_L2Maneuver`	scripted opponents (L1, L2): random turns and shots · precise tracking and shooting; based on the LAG base actor (`model_track`)
`model_SPManeuver`	self-play opponent (L3) with the pre-trained policy `sp_policy_name`
`model_strategy`	L4 opponent: mixed strategy over the pre-trained L3 (escape: L1) policies
`model_hier`	commander opponent with `opp_hier_strategy`
`model_human`	human pilot with joysticks or the keyboard via FlightSim (`MANUAL` in test.py); its F-16 has the corrected rudder of FlightSim's `Assets/Movers/JSBSimManual`

Interface	Definition
Low-level action	`MultiDiscrete([17, 11, 9, 9, 2])`: aileron, elevator, rudder, throttle, shoot
Low-level observation	30 values in [-5, 5]: own position, velocity, body rates, attitude, cannon state and ammunition; closest opponent: relative position, velocity and angles
Commander action	`Discrete(3)`: cannon, track, escape
Cannon	range 3500 m, cone ±10°, hit probability 0.8, shots = `num_missiles`; RL agents: a shot fires until it hits or for 12 s, the next one after 16 s (`launch()`); human: fires while the button is held, unlimited ammunition (`fire()`)

4.7 Training arguments (args.py)

Group	Arguments and defaults, set as flags, e.g. `python train.py --task track --level 2`
Run	`task track`, `level 1`, `hierarchy_level 1`, `run_name ""`, `restore_path None`, `seed 1`, `total_timesteps 5000000`
Algorithm	`algorithm ppo` (ppo, spo, mars), `trust_region_clip 0.25`, `gamma 0.99`, `td_lambda 0.92`, `use_gae True`, `entropy_coef 0.01`, `entropy_decay None`
Networks	`shared_policies True`, `centralized_critic True`, `actor_hidden_dim 128`, `critic_hidden_dim 128`, `actor_num_layers 1`, `critic_num_layers 1`, `use_ReLU True`, `normalize_input True`, `init_orthogonal True`
Recurrence, attention	`rnn False`, `rnn_num_layers 1`, `data_chunk_length 10`, `attention False`, `attention_heads 2` (rnn and attention exclude each other)
Optimization	`optimizer Adam`, `learning_rate_actor 3e-4`, `learning_rate_critic 3e-4`, `lr_decay None`, `train_epochs 10`, `batch_size 10000`, `minibatch_size 5000`, `clip_gradients 10`
Losses	`value_loss_coef 1.0`, `clip_value_loss 1.0`, `huber_coef 10.0`, `normalize_advantage True`, `normalize_value True`, `normalize_popart False`, `policy_mask True`, `value_mask True`
Workers, hardware	`num_envs 20`, `cuda True`, `cuda_id 0`, `torch_deterministic False`, `torch_compile False`, `avail_actions False`, `log_every_step 100`, `render_samples 4`

5 Quick start

Goal	Command
Set up FlightSim	`setup_windows.bat` · `./setup_linux.sh`
Try FlightSim	`.venv\Scripts\python Test_FlightSim_Windows.py` · `.venv/bin/python Test_FlightSim_Linux.py`
After C++ changes	`Tools\update_pythonbindings_windows.bat` · `./Tools/update_pythonbindings_linux.sh`
Train low level	`python train.py --task track --level 1`, then `--level 2 --restore_path runs/track_L1_f16_<run_name>`
Train commander	`python train.py --task hier --hierarchy_level 2`
Inference	set the settings in `test.py`, then `python test.py` (commander: `RESTORE_PATH` of a hier run and `--hierarchy_level 2`)
Record a video	press R in the FlightSim window to start and again to stop (or `app.start_recording()`); MP4 files in `recordings/`
Update this PDF	`.venv\Scripts\python Tools\build_instructions_pdf.py`

The venv of HHMARL_3D_NEW needs Python 3.11, like the `skybolt` module of FlightSim, and `imageio-ffmpeg` of its `requirements.txt` for the videos.